TECNICATURA SUPERIOR EN CIENCIA DE DATOS E INTELIGENCIA ARTIFICIAL

Centro Educativo Paulo Freire — U.E.G.P. N° 165

CLASE 14 — GUÍA PRÁCTICA

Implementación de K-Means en Python

Segmentación de clientes con scikit-learn

Datos de la clase

Duración del bloque práctico: **40 minutos** · Entorno: **Jupyter Notebook** · Librería: **scikit-learn**

Este material acompaña la teoría de la Clase 14 (Aprendizaje No Supervisado / Clustering I) y se resuelve de forma guiada en clase.

Objetivo del bloque práctico

Aplicar K-Means con scikit-learn sobre un dataset simulado de clientes, recorriendo el flujo completo: estandarización, entrenamiento, inspección de resultados y visualización. Luego, resolver tres ejercicios que profundizan en los puntos críticos vistos en la teoría: inicialización, elección de K y escalado de variables.

Flujo Guiado: Segmentación de Clientes

Trabajaremos con un dataset simulado de clientes con dos variables: `Ingresos_Anuales` y `Puntaje_Gasto`. El objetivo es identificar segmentos (grupos) de clientes con comportamientos similares, sin contar con etiquetas previas.

Paso 1 — Estandarización de variables

Como se vio en la teoría, K-Means depende de la distancia euclídea, por lo que es indispensable escalar las variables antes de entrenar el modelo:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(datos[['Ingresos_Anuales', 'Puntaje_Gasto']])
```

Paso 2 — Entrenamiento y ajuste del modelo

Instanciamos el modelo indicando el número de clusters K, la estrategia de inicialización y la cantidad de inicializaciones aleatorias a probar:

```
from sklearn.cluster import KMeans

# Ajustamos K-Means con K=4 y 20 inicializaciones aleatorias
kmeans = KMeans(n_clusters=4, init='k-means++', n_init=20, random_state=42)
kmeans.fit(X_scaled)
```

> **Nota**
> El parámetro `init='k-means++'` no es la inicialización totalmente aleatoria vista en la teoría: es una variante que distribuye los centroides iniciales de forma más inteligente, mejorando la convergencia. Aun así, seguir usando `n_init` alto sigue siendo la práctica recomendada.

Paso 3 — Inspección de atributos del modelo

Una vez entrenado, el objeto kmeans expone los resultados a través de tres atributos principales:

Atributo	Qué representa
`kmeans.labels_`	Vector con la etiqueta de grupo asignada a cada observación (0, 1, 2, 3).
`kmeans.cluster_centers_`	Coordenadas de los centroides finales, en el mismo espacio (escalado) que X_scaled.
`kmeans.inertia_`	Suma de cuadrados intra-cluster (la función objetivo minimizada). A menor valor, clusters más compactos.

Paso 4 — Visualización de clusters y centroides

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(8, 6))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=kmeans.labels_, cmap='viridis', s=50)
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
            c='red', marker='X', s=200, label='Centroides')
plt.title('Segmentación de Clientes con K-Means (K=4)')
plt.xlabel('Ingresos (Estandarizados)')
plt.ylabel('Puntaje de Gasto (Estandarizado)')
plt.legend()
plt.show()
```

El color de cada punto indica el cluster asignado; las X rojas marcan la posición final de los cuatro centroides.

Propuesta de Ejercicios Prácticos

Resolver en el notebook de la clase, en el orden propuesto. Cada ejercicio retoma uno de los puntos críticos del algoritmo señalados en el bloque teórico.

EJERCICIO 1 El Impacto de la Inicialización y Óptimos Locales

Objetivo del ejercicio

Verificar de forma empírica por qué n_init influye en la calidad del resultado final.

Consigna: Usando un dataset sintético generado con `make_blobs`:

1. Ajustar un modelo `KMeans(n_clusters=3, n_init=1, random_state=3)` y registrar el valor de `kmeans.inertia_`.
2. Ajustar un segundo modelo con `KMeans(n_clusters=3, n_init=20, random_state=3)` y comparar su inercia con la del anterior.
3. Responder la pregunta de reflexión.

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X, _ = make_blobs(n_samples=300, centers=3, cluster_std=1.5, random_state=3)

modelo_1 = KMeans(n_clusters=3, n_init=1, random_state=3)
modelo_1.fit(X)
print('Inercia con n_init=1: ', modelo_1.inertia_)

modelo_2 = KMeans(n_clusters=3, n_init=20, random_state=3)
modelo_2.fit(X)
print('Inercia con n_init=20:', modelo_2.inertia_)
```

Pregunta de reflexión

¿Por qué la inercia con n_init=20 suele ser menor o igual que con n_init=1? Explicar cómo influye la asignación inicial aleatoria de los centroides en la convergencia a un óptimo local, y por qué probar varias inicializaciones reduce el riesgo de quedar atrapado en una solución subóptima.

EJERCICIO 2 El Método del Codo (Elbow Method)

Objetivo del ejercicio

Aplicar el criterio del codo para elegir un número razonable de clusters cuando no se conoce K de antemano.

Consigna: Se entrega un dataset de clientes sin etiquetas.

1. Escribir un bucle `for` que ejecute K-Means variando K desde 1 hasta 10 clusters.
2. En cada iteración, almacenar el valor de `kmeans.inertia_` en una lista llamada `inercias`.
3. Graficar K en el eje horizontal vs. la inercia en el eje vertical.
4. Realizar el análisis pedido.

```
inercias = []
rango_k = range(1, 11)

for k in rango_k:
    modelo = KMeans(n_clusters=k, n_init=20, random_state=42)
    modelo.fit(X_scaled)
    inercias.append(modelo.inertia_)

plt.plot(range(1, 11), inercias, marker='o')
plt.xlabel('Número de clusters (K)')
plt.ylabel('Inercia')
plt.title('Método del Codo')
plt.show()
```

Análisis

Identificar visualmente el punto de "codo" (donde la curva se aplana). ¿Cuál sería el número óptimo de clusters para este problema de negocio? Justificar la elección en términos del compromiso entre simplicidad del modelo (menos clusters) y reducción de la inercia.

EJERCICIO 3 ¿Por qué es Vital Escalar las Características?

Objetivo del ejercicio

Comprobar de forma práctica el efecto de la escala de las variables sobre la distancia euclídea y, en consecuencia, sobre el resultado de K-Means.

Consigna: Imaginar datos de clientes con dos columnas: `Salario_Anual` (entre $20.000 y $150.000) y `Hijos` (entre 0 y 5).

1. Si se calcula la distancia euclídea entre dos clientes sin estandarizar, ¿qué variable determinará casi el 100% de la distancia? Justificar sin ejecutar código, a partir de la fórmula de distancia euclídea.
2. Aplicar K-Means sobre los datos crudos y luego sobre los datos transformados con `StandardScaler`.
3. Graficar ambos resultados y redactar una conclusión técnica.

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

X_crudo = datos[['Salario_Anual', 'Hijos']].values

# --- Sin escalar ---
kmeans_crudo = KMeans(n_clusters=3, n_init=20, random_state=42)
labels_crudo = kmeans_crudo.fit_predict(X_crudo)

# --- Escalado ---
X_std = StandardScaler().fit_transform(X_crudo)
kmeans_std = KMeans(n_clusters=3, n_init=20, random_state=42)
labels_std = kmeans_std.fit_predict(X_std)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].scatter(X_crudo[:, 0], X_crudo[:, 1], c=labels_crudo, cmap='viridis')
ax[0].set_title('Sin escalar')
ax[1].scatter(X_std[:, 0], X_std[:, 1], c=labels_std, cmap='viridis')
ax[1].set_title('Escalado con StandardScaler')
plt.show()
```

Conclusión a redactar

Comparar ambos gráficos: ¿los clusters obtenidos agrupan a los clientes de forma coherente en los dos casos, o solo en uno? Redactar una conclusión técnica sobre por qué el preprocesamiento de escala es un requisito indispensable en algoritmos basados en distancia como K-Means.

Cierre del bloque práctico

Los tres ejercicios cubren, en orden, los tres puntos críticos de K-Means vistos en la teoría: inicialización (n_init), elección de K (método del codo) y escalado de variables (StandardScaler). Guardar las conclusiones de cada ejercicio para la puesta en común.